

Atherneth: CS120 Project Report

Xinyue JIANG*, Ruoxuan ZHANG†

January 13, 2025

Abstract

In this project, we develop an audio-based computer network from the physical layer to the application layer using Rust. Initially, we established audio communication through air as the physical layer. To enhance the medium between two nodes, we introduced cables, improving signal transmission. At this stage, we utilized a mixer to collect audio signals and distribute them to their destinations based on MAC addresses. Subsequently, we incorporated IP addressing into the network, enabling IP address management. In the final phase, we aimed to allow a node to connect to a server via this audio network.

1 Introduction

This project explores the construction of a computer network using a bottom-up approach, covering the development from the physical layer to the network layer. Initially implemented in Java, the project was later transitioned to Rust to leverage its performance, safety, and concurrency benefits. This shift in programming languages reflects both the growing demand for Rust in system-level programming, and our goal optimizing the implementation for real-time audio processing.

The motivation behind this project was to investigate the feasibility of creating a functional network using unconventional communication channels, specifically audio signals. Starting at the physical layer, we establish communication through air, representing signals as sound waves transmitted between nodes. Gradually, we improved signal quality by introducing cable-based transmission, mimicking traditional wired networks. The implementation of a mixer at this stage facilitated efficient signal distribution to specific destinations based on Media Access Control (MAC) addresses.

As the project evolved, we extend the functionality by integrating Internet Protocol (IP) addressing, allowing devices within the network to be uniquely identified and communicate more effectively. The culmination of this work involved enabling a single node to connect to an external server using this custom-built audio network, demonstrating its application as a basic networking framework.

This project not only highlights the potential of audio as a communication medium, but also emphasizes the adaptability and power of Rust to develop efficient and safe network programming. [Yan24]

2 Project Technical Details

2.1 Project 0

At the beginning, we focused on setting up the development environment. Our team initially chose Java due to its robust support for audio communication, making it preferable to C++ in this aspect, and its better performance compared to Python. Lenovo has encapsulated its audio interface with proprietary technology, which leads to a conflict with ASIO. However, two weeks into the project, we faced significant challenges with hardware compatibility. Thanks to the guidance provided by wenxuanjun’s [JACK Instruction with Rust Example](#), we transitioned to [QJackCtl](#) as our audio driver and adopted Rust as the programming language. Although Rust enables the project to run more smoothly from a long-term perspective, Java is obviously easier to pick up, which also causes a significant hindrance to

*ShanghaiTech University, School of Information and Science Technology, 2022591003

†ShanghaiTech University, School of Information and Science Technology, 2021571004

project progress. At this stage, we focus on basic data transmission without strict time constraints. We planned to address timing and synchronization issues in subsequent phases.

2.2 Project 1

In Project 1 we work on the physical layer. So, we need to consider both the hardware and software. Plus, in air medium, it is plausible that errors occur more frequently, bits need to be transmitted with carrier waves. After sound wave generated, if wanted to transmit, we need to choose a proper modulation method as phase shift key modulation, The preamble code, which is a chirp signal helps each node to recognize the beginning of a frame of bits. However, to precisely capture the peak of the preamble part, we need to carefully move the threshold value.

Lastly, adding a Cyclic Redundancy Check will let the code to retransmit if not passed, and can use the time to compensate for the accuracy. For error correction, we choose [Reed-Solomon code](#), which will add some parity bits with original data bits. Then, with a decoder, we have the original signal. The choice of carrier frequency and signal parameters is crucial for optimal system performance. After extensive testing, our team has determined that a preamble length ranging from 100 to 200 elements (implemented as an array in our code) and a chirp signal frequency range from 1 to 100 are effective. Additionally, a sampling rate greater than 48,000 Hz is recommended to ensure accurate signal capture and processing. These parameters provide a solid foundation for achieving high performance and reliability in our communication system.

For Orthogonal Frequency Division Multiplexing, we use 2 different frequent waves as carriers to transmit the signal. And the specific number is 3000, 4000 Hz. The length of the preamble should be smaller than 500.

2.3 Project 2

Having tackled the challenges of air-based transmission, we transitioned to cable-based communication. Although cables eliminated medium-related issues such as interference, new challenges emerged, particularly at the mixer, where signals from multiple nodes could overlap and cause distortion. To enhance signal clarity, we implemented [4b/5b encoding](#), which translates every 4 bits of data into 5-bit code words. In this part, a method called repetition coding, each bit of data is sent three times and the receiver uses majority voting to determine the original bit. This method enhances reliability and error tolerance by allowing recovery of the original value even if one bit in the group is corrupted.

This approach prevents long runs of identical bits (e.g., consecutive 0s), which can cause receiver desynchronization. The added bits also ensured sufficient signal transitions for clock recovery. For the error correction part, we know that in theory, error correction codes could help address some of the issues, but as the number of bits and signals increased, correcting each error became less efficient. Error correction codes are inherently limited in their ability to correct only a small number of bit errors. When collisions occur, they often result in widespread errors that exceed the correction capacity of these codes. Thus, attempting to correct errors is actually impractical in the collision cases.

This phase focused on ensuring reliable signal transmission and synchronization between nodes within the network, laying the groundwork for the upcoming integration of higher-layer protocols.

When responding with an ACK frame, inserting the ACK at the beginning of the data payload can significantly accelerate transmission. This approach allows the receiver to quickly distinguish between the acknowledgment and the data payload, effectively "jumping over" the data part without waiting for the complete frame to process the acknowledgment. If we need a higher speed, this method provides an effective solution.

2.4 Project 3

With the physical and MAC layers established, the next step was to integrate the network layer. This phase introduced IP-based communication, allowing nodes within the network to send and receive data using Internet Protocol addresses.

Firstly, the need to reconstruct the code and incorporate IP functionality became evident, making asynchronous coding a necessity. At that juncture, [tokIO](#) emerged as a suitable choice. However when working with asynchronous programming, a multitude of issues can arise. For instance, after setting up the tokIO interface, a problem emerged where the Aethernet could only send data once and then

stopped functioning. Initially, the crossbeam method was attempted as a solution, but it proved to be incompatible. Instead, the mpsc (multi-producer, single-consumer) method was found to be more effective in resolving the issue. Although both methods can essentially achieve the same principle, the mpsc approach turned out to be the effective choice in this scenario. This case shows the challenge facing asynchronous condition. This was the [initial guide video](#) to which we adhered when commencing our work with Rust Async coding work, and we would highly recommend it to you. Additionally, there were instances where not all messages were being received. Implementing appropriate sleep intervals can potentially lead to improvements in message reception.

So at last we successfully implemented an ICMP Echo Request/Reply protocol. This allowed Node 1 to communicate with Node 2, which then connected to the Campus LAN to forward the echo request to a public server. The "aethernet" network now functioned as a bridge between the local network and the external internet. For this phase, we used a router configured as a basic web server to manage routing tasks. Advanced features like Network Address Translation (NAT) were implemented, with considerations for NAT traversal requiring an external device to enable seamless connections between nodes on different subnets.

2.5 Project 4

In this project, the communication has evolved from internal network interactions to engaging with external networks, introducing DNS and HTTP operations involving domain names, which were previously handled solely through IP addresses. This shift necessitates a more sophisticated approach to network management and testing, particularly in handling the complexities of domain resolution and data transfer over HTTP. At this juncture, the implementation of the TCP three-way handshake becomes exceedingly challenging. This is due to the increased complexity of managing connections between internal and external networks.

The primary goal of task 1 was to enhance the network capabilities by integrating the Domain Name System (DNS) and upgrading the Network Address Translation (NAT) to support both TCP and UDP protocols. This involved setting up a local DNS server on NODE2 and configuring NODE1 to resolve domain names using this server. So asynchronous I/O operations and TCP stream handling function need to be added. Notice that the use of a mutex (mutual exclusion) is essential in preventing data races in a multi-threaded or asynchronous environment. A mutex ensures that only one thread or task can access a shared resource at a time, thus maintaining data integrity and consistency.

In Task 2, which focuses on the HTTP aspect, the objective is to transfer textual information and display it on the terminal. When dealing with websites that have minimal content, it is highly probable that the data packets will be received in a sequential order. However, for websites with extensive content—comprising numerous words or a large volume of text—packet loss may occur. This issue typically arises when the web server's processing speed is slower than that of the transmitter, leading to the necessity for retransmission. On the receiving end, it is crucial to reorder the packets to reconstruct the correct sequence by examining their sequence numbers. Additionally, ACK (Acknowledgment) packets should be sent back to confirm receipt. During the testing period, it is crucial to pay close attention to a multitude of issues concerning firewalls, as they can significantly impact the network's access condition and the success of the tests.

3 Obstacles and Solutions

3.1 Environment Building

Initially, we encountered challenges in setting up the development environment for audio-based communication. While testing with our LENOVO laptop, we found significant compatibility issues between the ASIO audio driver and the hardware. This resulted in unstable performance and unexpected audio signal delays.

After several trials, we switched to QJackCtl, which proved to be a more stable and suitable audio driver for our project. It offered low-latency performance and supported the audio routing required for node-to-node communication. The switch marked a turning point in ensuring smooth signal transmission during subsequent project phases.

3.2 Language choice

After initially implementing the project in Java, we evaluated alternative programming languages for better performance and maintainability. Java provided robust libraries for audio communication but felt cumbersome and verbose for our needs.

We ultimately chose Rust due to its focus on speed, memory safety, and asynchronous programming capabilities. Rust's libraries enabled us to efficiently handle real-time audio data processing, and its syntax allowed for cleaner and more concise code compared to Java. Although a relatively young language, Rust proved its potential, particularly for network-related tasks.

For teams considering similar network projects, we recommend trying Rust for its excellent concurrency model and ecosystem of packages well-suited to asynchronous programming.

3.3 Attention of hardware problems

In the physical layer, transmission and reception issues often arise due to hardware limitations. Poor-quality headphones or microphones can struggle to play or capture high-quality audio signals, which hinders data integrity and increases error rates. Thus, upgrading to higher-quality audio devices can significantly improve signal fidelity and reduce debugging time. Investing in reliable hardware early on is a cost-effective strategy that saves copious hours of troubleshooting later. In Project 2, hardware variations play a significant role in system performance. Different computers operate at varying clock frequencies, which can lead to inconsistencies in data processing and transmission speeds. This variability may cause synchronization issues or uneven completion times across devices.

3.4 Testing parameter adjust

During Project 1 and 2, achieving high speed and accuracy required careful tuning of sampling rates. Higher rates, such as 48,000 Hz or even 96,000 Hz, offer faster data transfer but demand more processing power and better hardware capabilities. However, computers with limited performance may fail to operate reliably at high sampling rates, leading to increased errors and unstable communication.

Balance performance and hardware limitations by selecting the highest sampling rate your system can consistently support. If system limitations persist, consider offloading computation tasks to a more capable machine during critical testing phases.

3.5 Testing strategy

For the first two projects, assessment relied heavily on verifying the correctness and performance of audio transmissions. In practice, teaching assistants provided binary files as benchmarks for comparison. To deal with this case, developing testing scripts to automate transmission processes and validate outputs against the benchmarks in advance is a good choice. Use hash comparison tools in the terminal to quickly and reliably assess file integrity. Automating tests saves significant time and reduces the chance of human error, ensuring smoother progress during development. Additionally, the wave-display software featured on the project website, which is used to observe the waves collected at the physical layer, can significantly accelerate your debugging process.

3.6 Network firewall settings

Configuring firewalls to allow specific traffic is a critical task in network security and management. It ensures that only authorized and necessary data flows through the network, while blocking potentially harmful or unwanted traffic. This process, however, can be quite complex and requires careful attention to detail. For those who may find this task daunting, we write a [simple doc](#) that can provide some help on ICMP firewall block operation. And sometimes, errors can arise from firewall flaws, so be vigilant.

3.7 The long-term view in constructing program

In Project 2, we came to the realization that, at times, project requirements can be more efficiently tackled by adopting a long-term view. When working with larger sequences, it is possible to skip ahead to tasks that come later in the sequence. Instead of following each step one by one, this approach allows you to avoid wasting time on reworking or trimming tasks that come after. If you are confident about

the tasks further along, feel free to take on those challenges ahead of time, as it can save significant time in the process. Having a global perspective on the entire project is key to optimizing your workflow and achieving better results.

4 Conclusion and Suggestion

Throughout this project, we encountered various challenges that required both technical expertise and effective teamwork. At the outset, it was crucial to align programming language and technical flow. We had to ensure that the chosen tools, particularly the development environment and hardware, were compatible with the project's requirements. For instance, some computer models didn't perform well with certain drivers, so ensuring the hardware setup aligned with our needs was key. Having spend excessive time adapting to the environment would not have propelled our progress, so efficient hardware and software integration became one of our main priorities.

Project 1 taught us the importance of hardware quality. Although algorithms and theoretical models were crucial, many of the issues we encountered stemmed from hardware limitations, such as poor quality devices or inadequate processing power. Using better devices, such as high-quality microphones and headphones, improved overall performance and helped mitigate many issues that were initially perceived as algorithmic challenges.

Project 2 aims to provide a deeper understanding of the intricacies of network communication at the datalink layer and how various techniques can be applied to improve network performance and reliability.

Project 3 introduced us to Rust's tokIO crate, which provided an asynchronous runtime for handling multiple threads concurrently. This significantly improved the efficiency of our communication protocols and helped us scale the system to handle multiple tasks at the same time. The power of Rust, particularly to build reliable high-performance systems, was a key learning experience in this project.

In Project 4, the description of Task 5, [Browsing the Web](#), lacks clarity. Given that Task 2 already involves accessing HTML files over HTTPS, it would be more efficient in Task 5 to directly render the HTML content retrieved in Task 2, rather than relying on DNS traffic capture. This adjustment streamlines the workflow, reduces redundancy, and avoids unnecessary complexity, making the process more intuitive and efficient.

During our engagement with this course, we have identified several areas where the curriculum design could be enhanced for better learning outcomes. Specifically, the discussion periods and course structure seem to lack a transparent and impactful approach to facilitating our improvement.

For instance, the first discussion class on driver configuration was not so effective. Many students had already correctly configured their drivers, yet the course materials and instruction lagged behind, failing to provide timely and relevant guidance. This discrepancy between student progress and course pacing led to confusion and a lack of clarity on key concepts.

Furthermore, the second course module, which introduced simulations of routers and switches, aimed to test accessibility through ping commands. However, this module lacked continuity with the broader project objectives and course content. The exercise felt isolated. There is no other follow-up tasks or response.

To address these issues, we would like to suggest the following improvements:

1. **Enhanced Discussion Classes:** Increase the frequency and quality of discussion periods. These sessions should be more interactive, with clear objectives and outcomes. They should address more common issues and provide immediate feedback to students.
2. **Recitation Sessions:** Introduce regular recitation sessions that focus on problem-solving and hands-on practice. These sessions can help reinforce theoretical concepts with practical applications, ensuring that students can apply their knowledge effectively.
3. **Timely and Relevant Instruction:** Update the course content to align with the latest developments in network technology and student progress. This will prevent students from feeling ahead or behind the course curriculum.

However, the educational highlights of this course are undeniable. It has provided students with a broad perspective on the engineering aspects and offered an advanced introduction to the problem-solving mode that they will encounter in their future vocational settings. Embracing the challenge and getting our hands dirty is indeed a commendable starting point.

This project required intense teamwork and communication. Collaboration was essential as we had to align on decisions, share progress, and troubleshoot issues together. It also emphasized the importance of version control and software engineering principles. Treating the project from an engineering perspective meant that we had to plan, test, and refine our work in a structured and methodical way. Furthermore, Xinyue did an outstanding job with coding and version control, ensuring that the team remained organized and efficient throughout the process.

In conclusion, this has been a highly challenging course that provided valuable hands-on experience. Through practical application, we gained a deeper understanding of the concepts and techniques involved in building a network system.

References

- [Yan24] Zhice Yang. Course project documentation, fall 2024 — cs120 course project v0.86 documentation. <https://sist-cs120.github.io/project-doc/index.html>, 2024. Accessed: 2025-1-11.